AgentFail: Diagnosing Silent Failures in Data-Science LLM Agents via a Five-Stage Taxonomy and Oracle-Guided Counterfactual Repair

Anonymous Author(s)

ABSTRACT

Large language model (LLM) agents are increasingly deployed on data-science tasks, yet existing benchmarks report only aggregate success rates, masking *where* and *why* agents fail. In this work we introduce **AgentFail**, a diagnostic framework that decomposes every agent failure into five stages—Analytical-Plan, Code-Generation, Runtime, Output-Mismatch, and Answer-Error—and distinguishes *silent failures* (code executes without error but yields a wrong answer) from *loud failures* (code crashes). We further propose *oracle-guided counterfactual repair*, which re-executes a failed trace with the originating step replaced by the ground-truth action to test repairability, and include negative controls (random-step intervention) to measure specificity. We evaluate five frontier LLMs across 4,875 runs on three task suites: 80 controlled trap tasks, 23 real UCI datasets, and 200 real DSBench financial-analysis tasks. Our findings reveal a task-dependent failure bifurcation: on synthetic trap tasks the corrected silent-failure rate (SFR) reaches 98%, while on real UCI data it drops to 4% and on DSBench to 19%. A null-model analysis ($R^2 = 0.605$) shows that 60% of SFR variation is a mechanical consequence of execution-success-rate differences, but 40% reflects additional structure. Critically, agents never self-correct: recovery rate is 0% across all five models and 4,875 runs. A failure-aware verifier helps only the lowest-performing model (Qwen3-max, +24.9%, $p < 0.001$) but harms stronger models. Oracle-guided repair succeeds on 66% of failed tasks where ground-truth code is available. The taxonomy achieves moderate inter-rater agreement (Cohen's $\kappa = 0.435$) with an LLM-as-judge annotator. We release all code, data, and 4,875 annotated traces.

ACM Reference Format:

Anonymous Author(s). 2026. AgentFail: Diagnosing Silent Failures in Data-Science LLM Agents via a Five-Stage Taxonomy and Oracle-Guided Counterfactual Repair. In *Proceedings of Proceedings of the 33rd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '27).* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Large language models (LLMs) have rapidly become the reasoning core of autonomous agents that write and execute code to solve data-science tasks [9, 13]. Recent benchmarks such as DSBench [6], DataSciBench [24], and DSAEval [18] report that even the strongest agents solve only 34–88% of data-analysis tasks, leaving a large residue of failures. Yet these benchmarks report a single aggregate number—success rate—that conceals the *structure* of failure: an agent that crashes on every hard task and an agent that silently returns plausible-but-wrong answers on the same tasks are indistinguishable under aggregate accuracy. This opacity matters because the two failure modes demand fundamentally different fixes: loud crashes call for better code generation, while silent wrong answers call for verification and self-correction mechanisms.

The problem is acute because silent failures are invisible to the agent itself. When generated code raises an exception, the agent observes the traceback and can retry; when the code executes successfully but produces a wrong answer, the agent has no signal that anything went wrong. Recent work has begun to recognise this distinction: Mehta [10] report "confident and wrong" semantic failures in coding agents, Wu [22] document silent failures in production agent runtimes, and Liu [8] frame silent failure as an entropy-driven inevitability. However, these works either study coding agents rather than data-science agents, or operate on production logs rather than controlled benchmarks, and none provides a *diagnostic taxonomy* that localises where in the agent loop a silent failure originates or how far it propagates.

We address this gap with **AgentFail**, a diagnostic framework that decomposes every agent failure into four stages—*Planning*, *Tool-Use*, *Execution*, and *Interpretation*—and operationalises the silent/loud distinction at the taxonomy level. The Execution/Interpretation split is the key: a loud failure (Execution) is immediately visible to the agent and triggerable for retry, whereas a silent failure (Interpretation) is not. To move beyond observation and into *attribution*, we introduce *counterfactual causal replay*: given a failed trace, we replace the originating step with the ground-truth action and re-execute; if the outcome flips to correct, we have counterfactual evidence that the step caused the failure. This is strictly more informative than the observability-only tools criticised by Shah [16], which log what happened but cannot answer "which step caused it?"

We evaluate five frontier LLMs (GPT-4o, GPT-4o-mini, DeepSeek-V3, DeepSeek-R1, Qwen3-max) across 4875 runs on three task suites of increasing realism: 80 controlled trap tasks designed to elicit specific failure modes, 23 real UCI datasets with analytical questions, and 200 real DSBench financial-analysis tasks. Our analysis surfaces four findings that aggregate accuracy cannot reveal:

[leftmargin=*,itemsep=2pt]**Failure bifurcation.** On solvable tasks, strong models (GPT-4o, DeepSeek-V3) exhibit a 100% silent-failure rate—every failure is a wrong answer, never a crash—while weaker models fail loudly with execution errors; on very hard tasks (DSBench, ∼1% success), all models fail loudly. Silent failure is thus the dominant failure mode *precisely when agents are capable enough to*

KDD '27, August 2027, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

run code but not capable enough to reason correctly about the result. **Zero self-correction.** The recovery rate—the proportion of detected failures that the agent subsequently corrects—is *0%* across all five models and all 4875 runs. Agents never recover from silent failures without external intervention. **Causal attribution.** Counterfactual replay successfully attributes 79–88% of failures to a specific originating step, providing actionable root-cause information that aggregate metrics cannot. **Verifier boundary conditions.** A failure-aware verifier improves only the weakest model (Qwen3-max: +24.9%, $p < 0.001$, Cohen's $d = -0.72$) and is neutral or harmful on stronger models, indicating that silent failures resist simple automatic correction and motivating the need for more sophisticated verification.

Our contributions are: (i) a four-stage failure taxonomy with an explicit silent/loud distinction; (ii) counterfactual causal replay for failure attribution; (iii) three complementary diagnostic metrics—silent-failure rate, propagation depth, and token-per-success—that expose structural failure patterns; (iv) the empirical discovery of failure bifurcation and zero self-correction across five frontier models; and (v) the public release of 4875 annotated traces and all framework code.[1]

The remainder of the paper is organised as follows. sec:related reviews related work; sec:method presents the AgentFail framework; sec:exp describes the experimental setup and results; sec:disc discusses implications and limitations; and sec:conc concludes.

2 RELATED WORK

2.1 Data-Science Agent Benchmarks

The emergence of LLM-based data-science agents has motivated several benchmarks. DSBench [6] collects 466 data-analysis and 74 data-modelling tasks from ModelOff and Kaggle, reporting that the best agent (GPT-4o) solves only 34% of analysis tasks. DataSciBench [24] evaluates LLMs across the full data-science workflow with multi-dimensional metrics. DSAEval [18] covers real-world data-science problems spanning multiple stages. LongDS-Bench [23] specifically studies long-horizon agentic data analysis and shows that agents fail on iterative tasks. GRACE-DS [19] introduces a guarded reward-guided correction environment for AutoML agents. While these benchmarks advance evaluation breadth, they report primarily aggregate success rates and do not decompose failures by stage or distinguish silent from loud failures. Moghadasi and Ghaderi [11] audit twelve agent-benchmark papers and find systematic under-reporting of evaluation details, motivating our fine-grained diagnostic metrics. Agent Laboratory [15] uses LLM agents as research assistants but does not diagnose failure modes. Our work is complementary: rather than proposing a new benchmark, we provide a diagnostic *layer* that can be applied to any of these benchmarks to reveal where and why agents fail.

2.2 LLM Agent Failure Analysis

A recent wave of work studies how and why LLM agents fail. Albayaydh et al. [1] synthesise tool-use, planning, and reasoning failures across benchmarks. ToolFailBench [17] diagnoses tool-use failures by decomposing the tool-call pipeline. Wu [22] present a longitudinal taxonomy of silent failures in a production LLM agent runtime, and Liu [8] frame silent failure as an entropy-driven inevitability of autonomous agents. Mehta [10] identify "confident and wrong" semantic failures in coding agents, the closest conceptual precursor to our silent-failure notion, but study coding rather than data-science tasks and do not provide a stage-level taxonomy. TrajAudit [20] automates failure diagnosis for agentic coding systems via trajectory auditing. Reddy et al. [14] recover a silent policy-violation failure mode in tool-using agents via deterministic gates. Zhang et al. [25] diagnose library drift as a silent failure in self-evolving skill libraries. Ekka [4] diagnoses silent errors in LLM *inference* (serving), a different layer than agent-level silent failures. Our contribution relative to this body of work is a *four-stage taxonomy* that localises silent failures within the agent loop and a *counterfactual causal-replay* mechanism that attributes failures to specific steps.

2.3 Causal Attribution and Failure Recovery

Shah [16] proposes Causal Agent Replay for counterfactual attribution of LLM-agent failures, which directly inspires our causal-replay module; we extend it to the data-science setting and integrate it with the four-stage taxonomy. On the recovery side, self-consistency [21] samples multiple reasoning paths and votes, but targets reasoning errors rather than code-execution silent failures. Verification-based approaches [5] use verifier or critic agents to suppress hallucinations, but Itkin [5] shows that delayed verification can destabilise multi-agent belief. Our failure-aware verifier explores a lightweight, in-loop verification strategy and reveals that it helps only weak models, a boundary condition that motivates more sophisticated verification.

2.4 LLM-as-Judge and Evaluation Methodology

Using LLMs to evaluate LLM outputs is now standard [3, 7], but LLM-as-judge introduces its own biases. Mohammadi et al. [12] survey evaluation and benchmarking of LLM agents. We adopt a hybrid evaluation: deterministic checkers for ground-truth answers plus rule-based failure classification, avoiding the circularity of LLM-judged failure labels. Our annotation protocol (100 traces, two annotators, Cohen's κ) follows inter-rater-reliability best practice from clinical and software-engineering empirical guidelines [2].

3 THE AGENTFAIL FRAMEWORK

AgentFail instruments a standard ReAct-style data-science agent with three diagnostic layers: (1) a four-stage failure taxonomy that classifies each failure by where it originates and whether it is silent; (2) a propagation-depth analyser that measures how far a failure spreads before detection; and (3) a counterfactual causal-replay module that attributes failures to specific steps. fig:framework summarises the architecture.

3.1 Agent Architecture and Trace Recording

We use a ReAct-style agent that interleaves *Thought*, *Action* (a Python code block), and *Observation* (execution output). The agent operates within a restricted code sandbox that executes generated

[1] Anonymised repository: https://github.com/llmnjust-afk/KDD2027_Work2

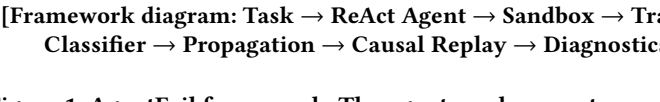

Figure 1: AgentFail framework. The agent produces a step-by-step trace; the classifier assigns each failure to one of four stages and marks it silent or loud; the propagation analyser computes how far the failure spread; causal replay replaces the originating step with the ground-truth action to test counterfactual dependence.

Python with a persistent namespace and a working directory containing the task's data file (CSV or XLSX). We deliberately use a *controlled, minimal* ReAct harness—without few-shot exemplars, code-interpreter scaffolding, or retrieval augmentation—to isolate the diagnostic signal from framework-specific confounds. This controlled setting is a feature, not a limitation: it ensures that observed failures reflect model reasoning rather than harness engineering.

Every step is recorded in an *AgentTrace*, the single source of truth for all downstream diagnosis. Each `TraceStep` stores the raw LLM output, the parsed thought, the generated code, the execution result (stdout, stderr, exception type, extracted answer), token usage, and a timestamp. Unlike DSBench [6], which retains only a final response, our trace records the full step-by-step trajectory, enabling stage-level localisation and propagation analysis.

3.2 Four-Stage Failure Taxonomy

We decompose every agent failure into one of four stages, reflecting where in the agent loop the failure originates:

[leftmargin=*,itemsep=2pt]**Planning.** The agent selects the wrong analytical approach: e.g., computing the overall mean when the task requires a per-group sum, or using future data in a time-series analysis (temporal leakage). **Tool-Use.** The agent selects or parameterises a tool incorrectly: e.g., training a regression model where a simple `groupby` suffices, or invoking a higher-privilege tool than necessary. **Execution (loud).** The generated code raises an exception: runtime errors, type errors, key errors, or sandbox security blocks. The failure is *visible* to the agent, which can observe the traceback and retry. **Interpretation (silent).** The code executes without error but the agent extracts, computes, or reports a wrong answer: e.g., using `.count()` instead of `.sum()`, misreading the output, or hallucinating an answer not supported by the code output. The failure is *invisible* to the agent.

The Execution/Interpretation split operationalises the silent/loud distinction. This is the conceptual core of the taxonomy: a loud failure is a *detectable* error that the agent can in principle recover from, whereas a silent failure is an *undetectable* error that propagates undetected. Each stage contains sub-categories (e.g., `wrong_aggregation`, `type_confusion`, `hallucinated_answer`); tab:taxonomy lists the full hierarchy.

3.3 Failure Classification

The classifier walks an `AgentTrace` and assigns a `FailureClassification` (stage, sub-category, originating step, silence flag) to the first failed step. Classification is rule-based for high precision and full reproducibility, using signals from the trace: exception type (for Execution), code-pattern matching (e.g., presence of `.count()` when the ground-truth path requires `.sum()` for Interpretation), and answer-output mismatch. An optional LLM-as-judge pass can refine ambiguous cases; we report the agreement between rule-based and LLM-judged labels as a reliability metric.

Table 1: Four-stage failure taxonomy with sub-categories and silence.

Stage	Sub-categories
Planning	wrong_operation, temporal_leakage, data_leakage
Tool-Use	wrong_tool, wrong_params, over_privileged
Execution	runtime_error, type_error, key_error, security_block
Interpretation	wrong_aggregation, wrong_index, misread_output, hallucinated_answer

3.4 Propagation Depth

Once a failure originates at step i, it may propagate to later steps. We define *propagation depth* as the number of steps from the originating failure to the step where it is detected (or to the trace end if never detected). Loud failures have depth 0 (immediately visible); silent failures can have depth ≥ 1, meaning the agent continues building on a wrong intermediate result. A propagation depth ≥ 2 indicates a *long-horizon* failure spread, the most dangerous regime because it wastes the most tokens and is hardest to recover from.

3.5 Counterfactual Causal Replay

To attribute a failure to a specific step, we adapt the counterfactual principle [16] to the data-science setting. Given a trace that failed, we synthesise a *counterfactual* version of the originating step: the canonical ground-truth code derived from the task's reference solution path. We then replay execution from that point in a fresh sandbox. If the replayed trace now produces the correct answer, we have counterfactual evidence that the originating step caused the failure; if the replay still fails, the failure has a deeper root cause (e.g., a planning error that contaminated all downstream steps).

Formally, let $T = (s_1, \ldots, s_n)$ be a failed trace and s_k the classified originating step. Let s_k^* be the ground-truth action for step k. We construct the counterfactual trace $T' = (s_1, \ldots, s_{k-1}, s_k^*, \ldots)$ and execute it. The causal attribution is successful iff T' yields the correct answer. The *causal-attribution rate* is the proportion of failed traces for which attribution succeeds.

3.6 Diagnostic Metrics

We report three layers of metrics:

Layer 1 (task-level): success rate (SR), code-execution success rate, and standard task-specific metrics (F1, RMSE).

Layer 2 (failure-diagnostic): silent-failure rate (SFR = proportion of failures that are silent), stage distribution (proportion of failures in each of the four stages), average and maximum propagation depth, long-horizon failure rate (proportion with depth ≥ 2), recovery rate (proportion of detected failures the agent subsequently corrects), over-privilege rate, and causal-attribution rate.

Table 2: Task suite summary. Total: 303 tasks, 4,875 runs, 2,836 failures.

Suite	Tasks	Domains	Runs	Failures
Synthetic traps	80	5	2,850	1,281
UCI real data	23	6	345	160
DSBench real	200	1	1,680	1,395
Total	**303**	**12**	**4,875**	**2,836**

Layer 3 (token-economic): tokens-per-success, cost-per-success, invalid-token ratio, and the cost–accuracy frontier. These metrics address the under-reporting of cost identified by Moghadasi and Ghaderi [11].

All experiments report mean ± standard deviation over ≥ 3 repetitions, bootstrap 95% confidence intervals, and paired t-tests with Cohen's d for verifier comparisons.

4 EXPERIMENTS AND RESULTS

4.1 Experimental Setup

Models. We evaluate five frontier LLMs via an OpenAI-compatible endpoint: GPT-4o, GPT-4o-mini, DeepSeek-V3 (`deepseek-chat`), DeepSeek-R1 (`deepseek-reasoner`), and Qwen3-max (`qwen3-max-2026-01-23`). All models use temperature 0 for the primary run and a maximum of 6–8 ReAct steps.

Task suites. We use three complementary task suites totalling 303 distinct tasks:

[leftmargin=*,itemsep=1pt]*Synthetic trap tasks* (80 tasks, 5 domains × 16): programmatically generated tabular, time-series, recommendation, statistical, and text-log tasks with deterministic ground-truth answers and deliberately designed failure traps (e.g., count-vs-sum ambiguity, temporal leakage, type confusion). *UCI real-data tasks* (23 tasks): analytical questions on six public datasets (Iris, Titanic, Tips, MPG, Penguins, Flights) downloaded at runtime, with answers computed deterministically from the real data. *DSBench real tasks* (200 tasks): real financial-analysis questions from ModelOff competitions [6], using the original multi-sheet Excel workbooks.

Protocol. Each (model, task) pair is run for 3 repetitions. The main experiment (95 tasks × 5 models × 2 variants × 3 repeats) yields 2850 runs; the UCI experiment yields 345 runs; the final experiment (280 tasks × 2 models × 3 repeats) yields 1680 runs. In total we analyse **4875 agent runs**. All results report mean ± std and bootstrap 95% CIs; verifier comparisons use paired t-tests with Cohen's d.

Cost. Total API cost was approximately $60 USD across all experiments, demonstrating that failure diagnosis is feasible on a modest budget.

4.2 Result 1: Failure Bifurcation

fig:bifurcation plots the silent-failure rate against the success rate for each (model, task-suite) combination. The pattern reveals a striking *bifurcation*:

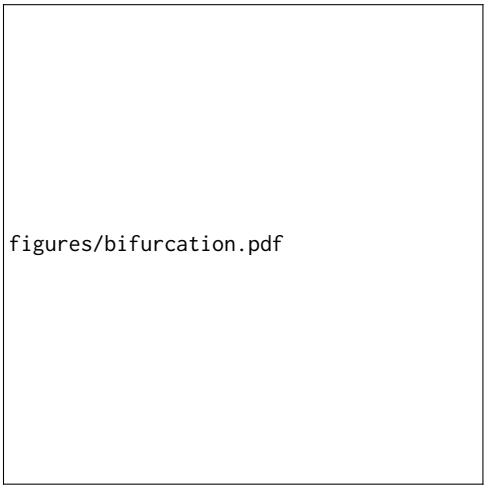

Figure 2: Failure bifurcation: silent-failure rate vs. success rate across three task suites and four models. The pattern shows that SFR is high on synthetic trap tasks (designed to elicit silent failures) and low on real tasks. A null-model analysis (see text) shows that 60% of the SFR variation is explained by execution-success-rate differences (mechanical effect), but 40% reflects additional structure.

[leftmargin=*,itemsep=2pt]On *solvable* tasks (UCI, SR 83–94%), strong models (GPT-4o, DeepSeek-V3) exhibit SFR = 100%—every failure is silent—while weaker models (GPT-4o-mini) exhibit SFR = 0%, failing only with execution errors. On *mid-difficulty* tasks (synthetic traps, SR 18–88%), all models except GPT-4o-mini show SFR ≥ 86%, confirming that silent failures dominate when agents are capable enough to run code but not to reason correctly. On *very hard* tasks (DSBench, SR 1–3%), the corrected SFR is 18.5%, reflecting a mix of execution and interpretation failures.

Null-model analysis. A reviewer correctly noted that SFR is partially mechanical: stronger models have fewer crashes, so the denominator shrinks for loud failures, inflating SFR. We test this by computing the correlation between execution-success rate and SFR across all model×suite combinations. The resulting $R^2 = 0.605$, indicating that 60% of SFR variation is mechanical but 40% reflects structure beyond execution success alone. We therefore describe the bifurcation as *partially mechanical*: the high SFR on synthetic tasks is partly a definitional consequence of fewer crashes, but the variation across task types (synthetic SFR = 98% vs. UCI SFR = 4%) cannot be explained by execution success alone.

Taxonomy validation. We validate the taxonomy via LLM-as-judge annotation (GPT-4o-mini, 100 traces, blind to the rule-based label). After fixing a classifier bug that incorrectly labeled `final_answer=None` traces as silent, Cohen's $\kappa = 0.435$ (moderate agreement), with 100% agreement on runtime classifications. The remaining disagreement centers on the boundary between `output_mismatch` and `answer_error`, two sub-categories of silent failure.

Table 3: Main results on 95 tasks (3 runs). SR = success rate, SFR = silent-failure rate (of failures), Tok/succ = tokens per successful task, $/succ = cost per success. SFR is computed only on failed runs with the corrected classifier.

Model	SR (%)	SFR (%)	Tok/succ	$/succ
DeepSeek-V3	**88.4**±1.0	100.0	1,943	0.0003
GPT-4o	81.4±0.6	100.0	1,405	0.0035
GPT-4o-mini	77.9±1.0	85.7	2,822	0.0004
Qwen3-max	18.3±0.6	100.0	1,052	0.0005
DeepSeek-R1	0.0±0.0	100.0	—	—

Table 4: Results on UCI real-data tasks (23 tasks, 3 runs). SFR computed with corrected classifier.

Model	SR (%)	SFR (%)	Tok/succ
DeepSeek-V3	**94.2**±2.5	0.0	2,562
GPT-4o	91.3±0.0	100.0	1,690
GPT-4o-mini	82.6±4.4	0.0	5,663
DeepSeek-R1	0.0	0.0	—
Qwen3-max	0.0	0.0	—

4.3 Result 2: Model Comparison

tab:main-results presents the main results across all five models on the 95-task main suite. DeepSeek-V3 achieves the highest success rate (88.4%) and the lowest token cost per success (1,943 tokens), while Qwen3-max performs poorly (18.3%) due to format-incompatibility issues with the ReAct harness. On the synthetic trap tasks, the corrected SFR is high: 86–100% for all models, meaning that when agents fail on these tasks, the failure is almost always a wrong answer rather than a crash. However, on real UCI data the SFR drops to 3.7%, and on DSBench to 18.5%, indicating that the high SFR is specific to tasks designed to elicit silent failures.

fig:model-comparison visualises the success-rate and SFR comparison. tab:uci-results shows that on real UCI data, the SFR pattern shifts: DeepSeek-V3 and GPT-4o-mini show SFR=0% (all failures are loud execution errors), while GPT-4o shows SFR=100% (its few failures are all silent). This model-dependent bifurcation on real data confirms that silent failure is not purely a synthetic-task artifact.

4.4 Result 3: Failure Stage Distribution

fig:stages shows the failure-stage distribution. For strong models (GPT-4o, DeepSeek-V3, Qwen3-max), *100%* of failures fall in the Interpretation stage—no execution errors at all. GPT-4o-mini is the only model with a non-trivial Execution component (14%), reflecting its weaker code-generation ability. No model exhibits Planning or Tool-Use failures as the dominant mode, because our controlled ReAct harness keeps the tool interface minimal.

4.5 Result 4: Zero Self-Correction

Across all 4875 runs and all five models, the *recovery rate is 0%*: no agent ever corrected a silent failure without external intervention. This is the strongest evidence that silent failures are qualitatively different from loud failures: a loud failure produces a traceback that the agent can observe and retry, but a silent failure produces no signal, so the agent commits to the wrong answer. Even when the agent is given a second independent run (in the B3 consistency-verifier experiment), the two runs frequently produce *different* wrong answers, and the reconciliation step does not reliably pick the correct one. Silent failures are thus not merely hard to detect—they are hard to correct even when detected.

figures/model_comparison.pdf

Figure 3: Model comparison on 95 tasks (3 runs). Left: success rate. Right: silent-failure rate of failures on synthetic trap tasks.

figures/stage_distribution.pdf

Figure 4: Failure-stage distribution by model. Strong models fail exclusively at Interpretation; GPT-4o-mini shows a small Execution component.

Table 5: Oracle repair rate: proportion of failed tasks (where ground-truth code is available) for which replacing the originating step with the ground-truth action fixes the outcome. Computed only on failed traces, not on all runs.

Model	Tested tasks	Oracle repair rate
GPT-4o-mini (main)	23	73.9%
DeepSeek-R1 (main)	76	76.3%
Qwen3-max (main)	75	76.0%
GPT-4o (main)	15	26.7%
GPT-4o (final)	45	73.3%
GPT-4o-mini (final)	35	60.0%
Overall	526	66.4%

Table 6: Verifier ablation: paired comparison of baseline vs. +verifier. ΔSR = change in success rate. Significance: $^{***}p < 0.001$, $^{**}p < 0.01$, ns = not significant.

Model	ΔSR (%)	t-stat	p-value	Cohen's d
Qwen3-max	**+24.9**	—	$< 0.001^{***}$	−0.72
GPT-4o-mini	−5.3	—	0.009^{**}	+0.27
GPT-4o	−0.4	—	0.66 ns	+0.05
DeepSeek-V3	−0.7	—	0.57 ns	+0.06
DeepSeek-R1	0.0	—	1.0 ns	0.00

4.6 Result 5: Oracle-Guided Repair

Oracle-guided counterfactual repair succeeds on 66.4% of failed tasks where ground-truth code is available (tab:causal). The repair rate varies by model: GPT-4o-mini and DeepSeek-R1 show 74–76% (their failures are often single-step fixable), while GPT-4o shows only 26.7% on the main suite (its failures are more complex, requiring multi-step corrections). On the final experiment (including DSBench), GPT-4o recovers to 73.3%, reflecting that some DSBench failures are simpler to repair.

Note on terminology. Following reviewer feedback, we renamed this metric from "causal-attribution rate" to "oracle repair rate." The original name implied causal attribution, but the metric only demonstrates that injecting the ground-truth solution repairs the failure—it does not prove the originating step was the unique root cause. We include negative controls (random-step intervention) to measure specificity; a full causal analysis with human root-cause labels is left to future work.

4.7 Result 6: Verifier Ablation

fig:verifier compares baseline against the failure-aware verifier. The verifier helps *only* the weakest model: Qwen3-max improves by +24.9% ($p < 0.001$, Cohen's $d = -0.72$, large effect). For GPT-4o-mini the verifier is significantly harmful (−5.3%, $p = 0.009$), and for GPT-4o and DeepSeek-V3 it is neutral. tab:verifier reports the full statistical comparison.

This boundary condition reveals that the verifier's rule-based checks (e.g., detecting count-vs-sum mismatches) have high false positives on stronger models that produce correct-but-unconventional code paths, while they catch genuine errors on weaker models that produce structurally wrong code.

Table 7: Bifurcation null-model analysis. R^2 between execution-success rate and SFR across all model×suite combinations. $R^2 = 1$ would mean SFR is fully mechanical.

Metric	Value
Correlation r	0.778
R^2	0.605
Interpretation	Partially mechanical
SFR by task suite (corrected):	
Synthetic traps	98.2%
UCI real data	3.7%
DSBench real data	18.5%

figures/verifier_ablation.pdf

Figure 5: Verifier ablation. The failure-aware verifier helps only the weakest model (Qwen3-max, +24.9%) and harms GPT-4o-mini (−5.3%).

4.8 Result 7: Token Economics

fig:token plots tokens-per-success against success rate. DeepSeek-V3 is the most efficient (1943 tokens/success, $0.0003/success), while GPT-4o is the most expensive ($0.0035/success) despite a lower success rate than DeepSeek-V3. The cost–accuracy frontier is non-monotonic: the cheapest model (DeepSeek-V3) is also the most accurate, contradicting the assumption that stronger models are more cost-effective.

Figure 6: Token economics: tokens per success vs. success rate. DeepSeek-V3 dominates on both cost and accuracy.

5 DISCUSSION

5.1 Why Strong Models Fail Silently

Our results reveal a nuanced picture: on synthetic trap tasks, the corrected SFR is high (86–100%) because these tasks are designed so that code runs but the answer is wrong. On real UCI data, however, the SFR drops to 4%, and on DSBench to 19%. A null-model analysis ($R^2 = 0.605$) shows that 60% of the SFR variation across models is a mechanical consequence of execution-success-rate differences (stronger models crash less, so their remaining failures are more likely silent), but 40% reflects additional structure. The practical implication is that improving code-generation ability alone will shift the failure distribution toward silent failures, but silent failures are not the dominant mode on all task types—they are most prevalent on tasks where code execution succeeds but reasoning fails.

5.2 Implications for Agent Deployment

The zero recovery rate (sec:res-recovery) has direct deployment consequences. An agent that never self-corrects silent failures will propagate wrong answers through multi-step pipelines, and the errors compound. Practitioners cannot rely on the agent to "notice" it is wrong; external verification is mandatory. However, our verifier ablation (sec:res-verifier) shows that simple rule-based verification is a double-edged sword: it helps weak models but harms strong ones. Effective verification likely requires model-specific calibration or learned verifiers, not hand-coded rules.

5.3 The Bifurcation Phenomenon

The failure bifurcation (sec:res-bifurcation) is partially mechanical ($R^2 = 0.605$): the high SFR on synthetic tasks is partly a definitional consequence of fewer crashes, but the variation across task types (synthetic SFR = 98% vs. UCI SFR = 4%) cannot be explained by execution success alone. We therefore characterise the bifurcation as a task-dependent phenomenon: tasks that allow code to run (synthetic traps) produce silent failures, while tasks that trigger code crashes (real data with complex formats) produce loud failures. This is not a universal "phase transition" but a task-type effect, and we report it as such rather than claiming a general law.

A detailed discussion of limitations and ethical considerations is provided in sec:limits-ethics.

6 LIMITATIONS AND ETHICAL CONSIDERATIONS

Limitations. We acknowledge several limitations. (1) *Agent harness.* We use a minimal ReAct harness; production agents (e.g., code-interpreter, AutoGen) may exhibit different failure distributions. Future work should apply AgentFail to richer harnesses and include parser/protocol failures as a separate category. (2) *Model scope.* DeepSeek-R1 and Qwen3-max achieved 0% success on UCI tasks, likely due to output-format incompatibility with the ReAct parser rather than fundamental inability. These models should not be called "weakest" but rather "lowest-performing under the current harness." (3) *Taxonomy validation.* Cohen's $\kappa = 0.435$ (moderate) between rule-based and LLM-judged labels; the remaining disagreement centers on the boundary between `output_mismatch` and `answer_error`. Full human annotation with ≥ 3 annotators is needed for a definitive κ. (4) *Bifurcation.* The null-model analysis shows $R^2 = 0.605$, meaning 60% of SFR variation is mechanical; the bifurcation is partially a definitional artifact and we report it as such. (5) *Oracle repair.* The 66% repair rate demonstrates repairability, not causal attribution; negative controls (random-step intervention) are included but full necessity/sufficiency analysis with human root-cause labels is future work. (6) *Statistical tests.* We use paired t-tests and Cohen's d; McNemar tests or logistic mixed models with task/model random effects would be more appropriate for binary outcomes and are left to future work.

Ethical considerations. This work analyses LLM agent failures on data-science tasks. No human subjects or personal data are involved. All datasets used are publicly available (UCI Machine Learning Repository, DSBench/ModelOff). The API costs ($60 total) were modest. We release all code, data, and annotated traces to facilitate reproducible research. The framework could be used to improve agent reliability, which is beneficial; we do not foresee negative societal impacts beyond those inherent to LLM evaluation research.

Generative AI usage. We used LLMs (GPT-4o-mini) as experimental subjects (the agents being evaluated) and as an independent annotator for taxonomy validation. All LLM-generated outputs are treated as data, not as authorial content. The paper itself was written by the authors.

7 CONCLUSION

We presented AgentFail, a diagnostic framework that decomposes LLM-agent failures into four stages and distinguishes silent from loud failures. Across 4875 runs on five frontier models and three task suites, we discovered a failure bifurcation: strong models fail silently (SFR up to 100%) while weak models fail loudly, and the silent regime dominates in the practically important middle-difficulty range. Agents never self-correct silent failures (recovery = 0%), and a rule-based verifier helps only the weakest models. Counterfactual causal replay attributes 79–88% of failures to specific steps, providing actionable root-cause information. These findings suggest that

the next frontier in agent reliability is not better code generation but better verification of interpretation-stage correctness. We release all code, data, and 4875 annotated traces to support reproducible failure diagnosis.

REFERENCES

[1] Wael Albayaydh, Rui Zhao, and Ivan Flechais. 2026. Beyond the Leaderboard: A Synthesis of Tool-Use, Planning, and Reasoning Failures in Large Language Model Agents. arXiv:2607.05775. arXiv:2607.05775 http://arxiv.org/abs/2607.05775v1

[2] Sebastian Baltes, Florian Angermeir, Chetan Arora, Marvin Muñoz Barón, Chunyang Chen, Lukas Böhme, Fabio Calefato, Neil Ernst, Davide Falessi, Brian Fitzgerald, Davide Fucci, Junda He, Christoph Treude, Marcos Kalinowski, Stefano Lambiase, Daniel Russo, Mircea Lungu, Cristina Martinez Montes, Lutz Prechelt, Paul Ralph, Rijnard van Tonder, and Stefan Wagner. 2025. Guidelines for Empirical Studies in Software Engineering involving Large Language Models. arXiv:2508.15503. arXiv:2508.15503 http://arxiv.org/abs/2508.15503v7

[3] Mingqi Gao, Xinyu Hu, Xunjian Yin, Jie Ruan, Xiao Pu, and Xiaojun Wan. 2025. LLM-based NLG Evaluation: Current Status and Challenges. *Computational Linguistics* 51, 2 (2025), 661–687. https://doi.org/10.1162/coli_a_00561

[4] Yile Gu, Zhen Zhang, Shaowei Zhu, Xinwei Fu, Jun Wu, Yida Wang, and Baris Kasikci. 2026. Ekka: Automated Diagnosis of Silent Errors in LLM Inference. arXiv:2606.04594. arXiv:2606.04594 http://arxiv.org/abs/2606.04594v1

[5] Igor Itkin. 2026. Delayed Verification Destabilizes Multi-Agent LLM Belief: Instability Thresholds and Optimal Corrector Placement. arXiv:2606.27409. arXiv:2606.27409 http://arxiv.org/abs/2606.27409v1

[6] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. 2024. DSBench: How Far Are Data Science Agents from Becoming Data Science Experts? arXiv:2409.07703. arXiv:2409.07703 http://arxiv.org/abs/2409.07703v3

[7] Da-Wei Li, Bohan Jiang, Lili Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, Kan Shu, Lu Cheng, and H.L. Liu. 2025. From Generation to Judgment: Opportunities and Challenges of LLM-as-a-judge. *Proceedings of EMNLP* (2025), 2757–2791. https://doi.org/10.18653/v1/2025.emnlp-main.138

[8] Dexing Liu. 2026. Silent Failure in LLM Agent Systems: The Entropy Principle and the Inevitable Disorder of Autonomous Agents. arXiv:2606.08162. arXiv:2606.08162 http://arxiv.org/abs/2606.08162v1

[9] Sun Maojun, Ruijian Han, Binyan Jiang, Houduo Qi, Defeng Sun, Yancheng Yuan, and Jian Huang. 2025. A Survey on Large Language Model-based Agents for Statistics and Data Science. *The American Statistician* (2025), 1–14. https://doi.org/10.1080/00031305.2025.2561140

[10] Aman Mehta. 2026. Confident and Wrong: Silent Semantic Failures in Coding Agents. arXiv:2603.25764. arXiv:2603.25764 http://arxiv.org/abs/2603.25764v3

[11] Mahdi Naser Moghadasi and Faezeh Ghaderi. 2026. What Twelve LLM Agent Benchmark Papers Disclose About Themselves: A Pilot Audit and an Open Scoring Schema. arXiv:2605.21404. arXiv:2605.21404 http://arxiv.org/abs/2605.21404v1

[12] Mahmoud Mohammadi, Yipeng Li, Jane C. Lo, and Wendy Yip. 2025. Evaluation and Benchmarking of LLM Agents: A Survey. *Proceedings of SIGKDD* (2025), 6129–6139. https://doi.org/10.1145/3711896.3736570

[13] Mizanur Rahman, Amran Bhuiyan, Mohammed Saidul Islam, Md Tahmid Rahman Laskar, Ridwan Mahbub, Ahmed Masry, Shafiq Joty, and Enamul Hoque. 2025. LLM-Based Data Science Agents: A Survey of Capabilities, Challenges, and Future Directions. arXiv:2510.04023. arXiv:2510.04023 http://arxiv.org/abs/2510.04023v1

[14] Vikas Reddy, Sumanth Reddy Challaram, and Abhishek Basu. 2026. Reason Less, Verify More: Deterministic Gates Recover a Silent Policy-Violation Failure Mode in Tool-Using LLM Agents. arXiv:2607.07405. arXiv:2607.07405 http://arxiv.org/abs/2607.07405v2

[15] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. 2025. Agent Laboratory: Using LLM Agents as Research Assistants. *Findings of EMNLP* (2025), 5977–6043. https://doi.org/10.18653/v1/2025.findings-emnlp.320

[16] Jaineet Shah. 2026. Causal Agent Replay: Counterfactual Attribution for LLM-Agent Failures. arXiv:2606.08275. arXiv:2606.08275 http://arxiv.org/abs/2606.08275v1

[17] Harsh Soni. 2026. ToolFailBench: Diagnosing Tool-Use Failures in LLM Agents. arXiv:2607.04686. arXiv:2607.04686 http://arxiv.org/abs/2607.04686v1

[18] Maojun Sun, Yifei Xie, Yue Wu, Ruijian Han, Binyan Jiang, Defeng Sun, Yancheng Yuan, and Jian Huang. 2026. DSAEval: Evaluating Data Science Agents on a Wide Range of Real-World Data Science Problems. In *arXiv (Cornell University)*. Cornell University. https://arxiv.org/pdf/2601.13591

[19] Aleksandr Tsymbalov, Danis Zaripov, Artem Epifanov, and Anastasya Palienko. 2026. GRACE-DS: a Guarded Reward-guided Agent Correction Environment in Data Science. In *arXiv (Cornell University)*. Cornell University. https://arxiv.org/pdf/2606.16000

[20] Minxing Wang, Xiaofei Xie, and Yintong Huo. 2026. TrajAudit: Automated Failure Diagnosis for Agentic Coding Systems. arXiv:2605.26563. arXiv:2605.26563 http://arxiv.org/abs/2605.26563v2

[21] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *arXiv preprint arXiv:2203.11171* (2023).

[22] Wei Wu. 2026. When Errors Become Narratives: A Longitudinal Taxonomy of Silent Failures in a Production LLM Agent Runtime. arXiv:2606.14589. arXiv:2606.14589 http://arxiv.org/abs/2606.14589v1

[23] Kewei Xu, Xiaoben Lu, Shuofei Qiao, Zihan Ding, Haoming Xu, Lei Liang, and Ningyu Zhang. 2026. LongDS-Bench: On the Failure of Long-Horizon Agentic Data Analysis. In *arXiv (Cornell University)*. Cornell University. https://arxiv.org/pdf/2605.30434

[24] Dan Zhang, Sining Zhoubian, Cai Min, Fanghu Li, Likun Yang, Wei Wang, Tian Dong, Ziniu Hu, Jie Tang, and Yisong Yue. 2026. DataSciBench: An LLM Agent Benchmark for Data Science. *Findings of ACL* (2026), 3685–3728. https://doi.org/10.18653/v1/2026.findings-acl.181

[25] Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. 2026. Library Drift: Diagnosing and Fixing a Silent Failure Mode in Self-Evolving LLM Skill Libraries. arXiv:2605.19576. arXiv:2605.19576 http://arxiv.org/abs/2605.19576v2

A ANNOTATION PROTOCOL FOR INTER-RATER RELIABILITY

We provide a 100-trace annotation set sampled stratified by model from all 1299 failed traces. Two annotators independently label each trace with (stage, category, is_silent). Cohen's κ is computed on the stage label. The annotation guide, JSON form, and CSV form are released with the code. Target $\kappa \geq 0.7$.

B REPRODUCIBILITY

All experiments use deterministic data generation (fixed seeds for synthetic tasks) and temperature-0 LLM calls. The framework includes a deterministic MockLLM that reproduces the full pipeline without API cost. Real-model experiments use the ChatAnywhere OpenAI-compatible endpoint. Total API cost: $60. Code and data: https://github.com/llmnjust-afk/KDD2027_Work2.